

Phần 7: Kết nối Thực tế và Best Practices

7.1 Transaction Management

Quản lý giao dịch (Transaction Management) là một khía cạnh cốt lõi trong các ứng dụng doanh nghiệp, đảm bảo tính toàn vẹn dữ liệu và độ tin cậy của các thao tác trên cơ sở dữ liệu. Spring cung cấp một cơ chế quản lý giao dịch mạnh mẽ và linh hoạt, cho phép bạn định nghĩa ranh giới giao dịch một cách khai báo (declarative) hoặc lập trình (programmatic).

7.1.1 @Transactional Configuration

Annotation `@Transactional` là cách phổ biến nhất để quản lý giao dịch trong Spring. Khi một phương thức được đánh dấu `@Transactional`, Spring sẽ tự động bắt đầu một giao dịch trước khi phương thức được thực thi và commit hoặc rollback giao dịch sau khi phương thức hoàn thành, tùy thuộc vào kết quả của phương thức.

Java

```
@Service // Đánh dấu lớp này là một Spring Service component.
public class UserService {

    @Autowired // Tự động inject UserRepository và EmailService.
    private UserRepository userRepository;

    @Autowired
    private EmailService emailService;

    // Default transaction: Phương thức này sẽ chạy trong một transaction.
    // Nếu không có transaction nào đang tồn tại, một transaction mới sẽ được tạo.
    // Nếu có, nó sẽ tham gia vào transaction hiện có.
    @Transactional
    public User createUser(User user) {
        User savedUser = userRepository.save(user); // Lưu người dùng vào cơ
        // sở dữ liệu.
        emailService.sendWelcomeEmail(savedUser.getEmail()); // Gửi email
        // chào mừng. Nếu có lỗi ở đây, toàn bộ transaction (bao gồm cả việc lưu user)
        // sẽ bị rollback.
        return savedUser;
    }

    // Read-only transaction (optimization): Đánh dấu transaction là chỉ đọc.
    // Điều này giúp tối ưu hóa hiệu suất vì cơ sở dữ liệu có thể bỏ qua việc quản
    // lý các lock ghi và các cơ chế khác liên quan đến việc thay đổi dữ liệu. Rất
```

hữu ích cho các phương thức chỉ đọc dữ liệu.

```
@Transactional(readOnly = true)
public List<User> getAllActiveUsers() {
    return userRepository.findByStatus("ACTIVE"); // Truy vấn tất cả
    người dùng có trạng thái ACTIVE.
}

// Custom transaction settings: Cấu hình chi tiết cho transaction.
@Transactional(
    propagation = Propagation.REQUIRES_NEW, // Luôn bắt đầu một
    transaction mới. Nếu có transaction hiện tại, nó sẽ bị tạm dừng.
    isolation = Isolation.READ_COMMITTED, // Mức độ cô lập (isolation
    level) của transaction. READ_COMMITTED ngăn chặn việc đọc dữ liệu chưa được
    commit bởi các transaction khác.
    timeout = 30, // Thời gian tối đa (giây) mà transaction có thể chạy
    trước khi bị rollback tự động.
    rollbackFor = {Exception.class}, // Chỉ định các loại Exception mà
    khi xảy ra, transaction sẽ bị rollback. Mặc định là RuntimeException và
    Error.
    noRollbackFor = {IllegalArgumentException.class} // Chỉ định các loại
    Exception mà khi xảy ra, transaction sẽ KHÔNG bị rollback.
)
public void complexBusinessOperation(Long userId) {
    // Complex business logic
    User user = userRepository.findById(userId).orElseThrow(); // Tìm
    người dùng theo ID, nếu không tìm thấy sẽ ném ngoại lệ.
    // ... business logic
    // Các thao tác thay đổi dữ liệu ở đây sẽ được thực hiện trong
    transaction mới này.
}

// Programmatic transaction: Quản lý transaction bằng code. Ít được sử
    dụng hơn @Transactional nhưng hữu ích trong các trường hợp phức tạp hoặc khi
    cần kiểm soát chi tiết hơn.
@Autowired
private PlatformTransactionManager transactionManager; // Inject
PlatformTransactionManager để quản lý transaction.

public void programmaticTransaction() {
    TransactionDefinition def = new DefaultTransactionDefinition(); //
    Định nghĩa các thuộc tính của transaction.
    TransactionStatus status = transactionManager.getTransaction(def); //
    Bắt đầu transaction và lấy về trạng thái của nó.

    try {
        // Business logic
        userRepository.save(new User()); // Thực hiện thao tác lưu người
        dùng.
    }
```

```

        transactionManager.commit(status); // Commit transaction nếu mọi
        thứ thành công.
    } catch (Exception e) {
        transactionManager.rollback(status); // Rollback transaction nếu
        có bất kỳ lỗi nào xảy ra.
        throw e; // Ném lại ngoại lệ để thông báo lỗi.
    }
}
}

```

7.1.2 Transaction Propagation

Transaction Propagation (lan truyền giao dịch) định nghĩa cách các phương thức được đánh dấu `@Transactional` hoạt động khi chúng được gọi từ một phương thức khác cũng được đánh dấu `@Transactional`. Nó xác định liệu một transaction mới có nên được tạo, hay phương thức nên tham gia vào một transaction hiện có, hay chạy mà không có transaction nào.

Java

```

@Service
public class OrderService {

    @Autowired
    private OrderRepository orderRepository;

    @Autowired
    private AuditService auditService;

    @Transactional // Transaction chính cho việc xử lý đơn hàng.
    public void processOrder(Order order) {
        orderRepository.save(order);

        // Audit sẽ chạy trong cùng transaction với processOrder
        (Propagation.REQUIRED là mặc định).
        auditService.logOrderCreated(order.getId());

        // Notification sẽ chạy trong transaction riêng
        (Propagation.REQUIRES_NEW).
        auditService.sendNotification(order.getId());
    }
}

@Service
public class AuditService {

```

```
@Autowired
private AuditRepository auditRepository;
```

// Chạy trong transaction hiện tại: Nếu có transaction đang tồn tại, phương thức này sẽ tham gia vào transaction đó. Nếu không, một transaction mới sẽ được tạo. Đây là hành vi mặc định và phổ biến nhất.

```
@Transactional(propagation = Propagation.REQUIRED)
public void logOrderCreated(Long orderId) {
    auditRepository.save(new AuditLog("ORDER_CREATED", orderId));
}
```

// Chạy trong transaction mới: Luôn bắt đầu một transaction mới cho phương thức này. Nếu có transaction hiện tại, nó sẽ bị tạm dừng cho đến khi transaction mới này hoàn thành.

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void sendNotification(Long orderId) {
    // Notification logic - ví dụ: gửi email. Nếu việc gửi email thất
    // bại, nó sẽ không ảnh hưởng đến transaction chính của OrderService.
    // Các thao tác DB trong phương thức này sẽ được commit/rollback độc
    // lập.
}
```

// Không tham gia transaction: Phương thức này sẽ chạy mà không có bất kỳ transaction nào. Nếu có transaction hiện tại, nó sẽ bị tạm dừng.

```
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void logSystemEvent(String event) {
    // System logging - ví dụ: ghi log vào file hoặc một DB không cần
    // transaction. Các thao tác DB ở đây sẽ không được quản lý bởi transaction.
}
}
```

Giải thích các loại Propagation:

- **REQUIRED (Default):** Nếu có transaction đang tồn tại, phương thức sẽ tham gia vào transaction đó. Nếu không, một transaction mới sẽ được tạo. Đảm bảo rằng phương thức luôn chạy trong một transaction.
- **SUPPORTS :** Nếu có transaction đang tồn tại, phương thức sẽ tham gia vào transaction đó. Nếu không, phương thức sẽ chạy mà không có transaction. Không bắt buộc phải có transaction.
- **MANDATORY :** Phương thức phải chạy trong một transaction hiện có. Nếu không có transaction nào đang tồn tại, một `TransactionRequiredException` sẽ được ném ra.

- **REQUIRES_NEW** : Luôn bắt đầu một transaction mới cho phương thức này. Nếu có transaction hiện tại, nó sẽ bị tạm dừng cho đến khi transaction mới này hoàn thành. Đảm bảo tính độc lập của transaction.
- **NOT_SUPPORTED** : Phương thức sẽ chạy mà không có transaction. Nếu có transaction hiện tại, nó sẽ bị tạm dừng. Hữu ích cho các thao tác không cần transaction (ví dụ: đọc dữ liệu không quan trọng, ghi log).
- **NEVER** : Phương thức không được phép chạy trong một transaction. Nếu có transaction đang tồn tại, một `IllegalTransactionStateException` sẽ được ném ra.
- **NESTED** : Nếu có transaction hiện tại, nó sẽ tạo một savepoint. Nếu phương thức thất bại, nó sẽ rollback về savepoint đó mà không ảnh hưởng đến transaction bên ngoài. Nếu không có transaction hiện tại, nó sẽ hoạt động như **REQUIRED** . (Chỉ hoạt động với một số loại transaction manager nhất định, ví dụ: JDBC).

7.2 Exception Handling

Quản lý ngoại lệ (Exception Handling) là một phần quan trọng của việc xây dựng các ứng dụng mạnh mẽ và đáng tin cậy. Trong ứng dụng Spring Boot với Spring Data JPA, việc xử lý ngoại lệ đúng cách giúp cung cấp phản hồi rõ ràng cho người dùng và duy trì tính toàn vẹn của hệ thống.

7.2.1 Custom Exceptions

Trong nhiều trường hợp, các ngoại lệ tiêu chuẩn của Java hoặc Spring không đủ để thể hiện các lỗi nghiệp vụ cụ thể của ứng dụng. Việc tạo các ngoại lệ tùy chỉnh (Custom Exceptions) giúp mã nguồn rõ ràng hơn, dễ bảo trì hơn và cho phép bạn xử lý các lỗi nghiệp vụ một cách có tổ chức.

Java

```
// Business exceptions: Các ngoại lệ tùy chỉnh nên kế thừa từ  
RuntimeException để không cần phải khai báo throws trong chữ ký phương thức  
(unchecked exceptions).  
public class UserNotFoundException extends RuntimeException {  
    public UserNotFoundException(String message) {  
        super(message);  
    }  
}
```

```

    public UserNotFoundException(Long userId) {
        super("User not found with ID: " + userId); // Constructor tiện lợi
        // để tạo thông báo lỗi cụ thể.
    }
}

public class DuplicateEmailException extends RuntimeException {
    public DuplicateEmailException(String email) {
        super("User with email already exists: " + email);
    }
}

// Service layer: Nơi các ngoại lệ nghiệp vụ thường được ném ra.
@Service
@Transactional // Đảm bảo các thao tác trong service layer chạy trong một
transaction.
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public User getUserById(Long id) {
        // Sử dụng orElseThrow để ném UserNotFoundException nếu không tìm
        // thấy người dùng.
        return userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
    }

    public User createUser(User user) {
        // Kiểm tra email trùng lặp trước khi lưu.
        if (userRepository.existsByEmail(user.getEmail())) {
            throw new DuplicateEmailException(user.getEmail());
        }

        try {
            return userRepository.save(user);
        } catch (DataIntegrityViolationException e) {
            // Xử lý trường hợp ngoại lệ DataIntegrityViolationException
            // (thường xảy ra khi có ràng buộc UNIQUE trong DB).
            // Kiểm tra nguyên nhân gốc (cause) của ngoại lệ để xác định lỗi
            // cụ thể.
            if (e.getCause() instanceof ConstraintViolationException) {
                throw new DuplicateEmailException(user.getEmail());
            }
            throw e; // Ném lại ngoại lệ nếu không phải lỗi trùng lặp email.
        }
    }
}

```

```
}  
}
```

Giải thích:

- **RuntimeException** : Các ngoại lệ nghiệp vụ thường kế thừa `RuntimeException` để chúng là các ngoại lệ không được kiểm tra (unchecked exceptions). Điều này giúp giữ cho mã nguồn sạch sẽ hơn vì bạn không cần phải khai báo `throws` trong chữ ký phương thức.
- **Constructor tiện lợi**: Việc tạo các constructor tùy chỉnh cho phép bạn truyền các thông tin cụ thể về lỗi (ví dụ: ID người dùng không tìm thấy, email trùng lặp) vào thông báo ngoại lệ.
- **orElseThrow()** : Một phương thức tiện lợi của `Optional` trong Java 8+, cho phép bạn ném một ngoại lệ tùy chỉnh nếu `Optional` rỗng.
- **DataIntegrityViolationException** : Đây là một ngoại lệ của Spring Data (kế thừa từ `RuntimeException`) được ném ra khi có vi phạm ràng buộc toàn vẹn dữ liệu trong cơ sở dữ liệu (ví dụ: ràng buộc `UNIQUE` , `NOT NULL` , `FOREIGN KEY`).
- **ConstraintViolationException** : Đây là một ngoại lệ của JPA/Hibernate, thường là nguyên nhân gốc (cause) của `DataIntegrityViolationException` khi có vi phạm ràng buộc cơ sở dữ liệu. Việc kiểm tra `e.getCause()` giúp bạn phân biệt các loại vi phạm ràng buộc khác nhau và ném ra ngoại lệ nghiệp vụ phù hợp.

7.2.2 Global Exception Handler

Để xử lý các ngoại lệ một cách tập trung và nhất quán trong ứng dụng Spring Boot, bạn có thể sử dụng `@RestControllerAdvice` (hoặc `@ControllerAdvice`) kết hợp với `@ExceptionHandler` . Điều này cho phép bạn định nghĩa các phương thức xử lý ngoại lệ toàn cục, chuyển đổi các ngoại lệ thành các phản hồi HTTP có ý nghĩa cho client.

Java

```
@RestControllerAdvice // Annotation này kết hợp @ControllerAdvice và  
@ResponseBody, cho phép xử lý ngoại lệ và trả về JSON/XML.  
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(UserNotFoundException.class) // Xử lý ngoại lệ
```

UserNotFoundException.

`@ResponseStatus(HttpStatus.NOT_FOUND)` // Đặt mã trạng thái HTTP là 404
Not Found.

```
public ErrorResponse handleUserNotFound(UserNotFoundException e) {  
    return new ErrorResponse("USER_NOT_FOUND", e.getMessage()); // Trả về  
    một đối tượng ErrorResponse tùy chỉnh.  
}
```

`@ExceptionHandler(DuplicateEmailException.class)` // Xử lý ngoại lệ
DuplicateEmailException.

`@ResponseStatus(HttpStatus.CONFLICT)` // Đặt mã trạng thái HTTP là 409
Conflict.

```
public ErrorResponse handleDuplicateEmail(DuplicateEmailException e) {  
    return new ErrorResponse("DUPLICATE_EMAIL", e.getMessage());  
}
```

`@ExceptionHandler(DataIntegrityViolationException.class)` // Xử lý ngoại
lệ *DataIntegrityViolationException.*

`@ResponseStatus(HttpStatus.BAD_REQUEST)` // Đặt mã trạng thái HTTP là 400
Bad Request.

```
public ErrorResponse  
handleDataIntegrityViolation(DataIntegrityViolationException e) {  
    // Trong môi trường production, bạn có thể không muốn hiển thị chi  
    tiết lỗi DB cho client.  
    return new ErrorResponse("DATA_INTEGRITY_VIOLATION", "Data constraint  
violation");  
}
```

`@ExceptionHandler(OptimisticLockingFailureException.class)` // Xử lý ngoại
lệ khi có xung đột *Optimistic Locking.*

`@ResponseStatus(HttpStatus.CONFLICT)` // Đặt mã trạng thái HTTP là 409
Conflict.

```
public ErrorResponse  
handleOptimisticLocking(OptimisticLockingFailureException e) {  
    return new ErrorResponse("CONCURRENT_MODIFICATION", "Resource was  
modified by another user");  
}
```

`@ExceptionHandler(MethodArgumentNotValidException.class)` // Xử lý ngoại
lệ khi validation thất bại (ví dụ: `@Valid` trên DTO).

`@ResponseStatus(HttpStatus.BAD_REQUEST)` // Đặt mã trạng thái HTTP là 400
Bad Request.

```
public ErrorResponse handleValidation(MethodArgumentNotValidException e)  
{  
    Map<String, String> errors = new HashMap<>();  
    // Lấy tất cả các lỗi validation từ BindingResult và đưa vào Map.  
    e.getBindingResult().getFieldErrors().forEach(error ->  
        errors.put(error.getField(), error.getDefaultMessage()));  
}
```



```

        return new ErrorResponse("VALIDATION_ERROR", "Validation failed",
errors); // Trả về thông tin chi tiết về các lỗi validation.
    }
}

// Error response DTO: Định nghĩa cấu trúc phản hồi lỗi cho client.
public class ErrorResponse {
    private String code; // Mã lỗi nội bộ.
    private String message; // Thông báo lỗi thân thiện với người dùng.
    private Map<String, String> details; // Chi tiết lỗi (ví dụ: lỗi
validation cho từng trường).
    private LocalDateTime timestamp; // Thời gian xảy ra lỗi.

    // constructors, getters, setters
    public ErrorResponse(String code, String message) {
        this.code = code;
        this.message = message;
        this.timestamp = LocalDateTime.now();
    }

    public ErrorResponse(String code, String message, Map<String, String>
details) {
        this.code = code;
        this.message = message;
        this.details = details;
        this.timestamp = LocalDateTime.now();
    }

    public String getCode() {
        return code;
    }

    public String getMessage() {
        return message;
    }

    public Map<String, String> getDetails() {
        return details;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }
}

```

Giải thích:

- **@RestControllerAdvice** : Annotation này được sử dụng để định nghĩa một lớp xử lý ngoại lệ toàn cục cho các controller. Nó cho phép bạn tập trung logic xử lý ngoại lệ ở một nơi duy nhất, thay vì phải xử lý trong từng controller riêng lẻ.
- **@ExceptionHandler(ExceptionType.class)** : Đánh dấu một phương thức để xử lý một loại ngoại lệ cụ thể. Khi một ngoại lệ thuộc loại này (hoặc lớp con của nó) được ném ra từ bất kỳ controller nào, phương thức này sẽ được gọi.
- **@ResponseStatus(HttpStatus.CODE)** : Đặt mã trạng thái HTTP cho phản hồi. Điều này rất quan trọng để client có thể hiểu được loại lỗi đã xảy ra (ví dụ: 404 Not Found, 409 Conflict, 400 Bad Request).
- **ErrorResponse DTO**: Việc định nghĩa một đối tượng Data Transfer Object (DTO) để trả về thông tin lỗi giúp đảm bảo phản hồi lỗi có cấu trúc nhất quán và dễ dàng được client phân tích cú pháp. Nó thường bao gồm một mã lỗi, thông báo lỗi và có thể là các chi tiết bổ sung.
- **MethodArgumentNotValidException** : Ngoại lệ này được ném ra khi validation của các đối tượng `@RequestBody` hoặc `@ModelAttribute` thất bại (ví dụ: khi sử dụng `@Valid` trên DTO đầu vào và có các ràng buộc validation như `@NotBlank` , `@Email` , `@Min` bị vi phạm).
- **BindingResult** : Đối tượng này chứa tất cả các lỗi validation khi `MethodArgumentNotValidException` xảy ra. Bạn có thể lặp qua `getFieldErrors()` để lấy thông tin chi tiết về lỗi cho từng trường.

7.3 Service Layer Design

Service Layer (lớp dịch vụ) là một thành phần quan trọng trong kiến trúc ứng dụng, đặc biệt là trong các ứng dụng lớn và phức tạp. Nó đóng vai trò là cầu nối giữa lớp Controller (hoặc Presentation) và lớp Repository (hoặc Data Access), chứa đựng logic nghiệp vụ chính của ứng dụng. Việc thiết kế Service Layer tốt giúp tách biệt các mối quan tâm (separation of concerns), tăng khả năng tái sử dụng code, dễ dàng kiểm thử và bảo trì.

7.3.1 Service Layer Structure

Một Service Layer điển hình thường bao gồm:

- **DTOs (Data Transfer Objects):** Các đối tượng dùng để truyền dữ liệu giữa các lớp (ví dụ: từ Controller đến Service, hoặc từ Service đến Controller). DTO giúp tách biệt mô hình dữ liệu của API khỏi mô hình Entity của cơ sở dữ liệu, cho phép bạn kiểm soát chính xác dữ liệu nào được hiển thị hoặc chấp nhận từ client.
- **Service Interface:** Định nghĩa các phương thức nghiệp vụ mà Service cung cấp. Điều này giúp tách biệt giao diện khỏi implementation, cho phép bạn thay đổi implementation mà không ảnh hưởng đến các thành phần sử dụng Service.
- **Service Implementation:** Chứa logic nghiệp vụ thực tế, sử dụng các Repository để tương tác với cơ sở dữ liệu và các Mapper để chuyển đổi giữa Entity và DTO.

Java

```
// DTO classes: Các lớp DTO thường không chứa logic nghiệp vụ, chỉ chứa dữ
liệu và các ràng buộc validation (nếu có).
public class CreateUserRequest {
    @NotBlank(message = "Tên không được để trống") // Ràng buộc validation
cho trường name.
    private String name;

    @Email(message = "Email không hợp lệ") // Ràng buộc validation cho trường
email.
    @NotBlank(message = "Email không được để trống")
    private String email;

    @Min(value = 18, message = "Tuổi phải lớn hơn hoặc bằng 18") // Ràng buộc
validation cho trường age.
    private Integer age;

    // getters, setters
}

public class UpdateUserRequest {
    private String name;
    private Integer age;
    // Chỉ các trường có thể được cập nhật mới có trong DTO này.
}

public class UserResponse {
    private Long id;
    private String name;
    private String email;
    private Integer age;
    private String status;
```

```

    private LocalDateTime createdAt;

    // constructors, getters, setters
}

// Service interface: Định nghĩa các hợp đồng (contract) của Service.
public interface UserService {
    UserResponse createUser(CreateUserRequest request);
    UserResponse updateUser(Long id, UpdateUserRequest request);
    UserResponse getUserById(Long id);
    Page<UserResponse> getUsers(String name, String status, Pageable
pageable);
    void deleteUser(Long id);
    void activateUser(Long id);
    void deactivateUser(Long id);
}

// Service implementation: Chứa logic nghiệp vụ thực tế.
@Service // Đánh dấu lớp này là một Spring Service component.
@Transactional // Đảm bảo các phương thức trong service này chạy trong một
transaction.
@Validated // Kích hoạt validation cho các phương thức của service (nếu có
các ràng buộc validation trên tham số phương thức).
public class UserServiceImpl implements UserService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private UserMapper userMapper; // Inject UserMapper để chuyển đổi giữa
Entity và DTO.

    @Override
    public UserResponse createUser(@Valid CreateUserRequest request) { //
@Valid kích hoạt validation cho request DTO.
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new DuplicateEmailException(request.getEmail());
        }

        User user = userMapper.toEntity(request); // Chuyển đổi DTO sang
Entity.
        user.setStatus("ACTIVE");

        User savedUser = userRepository.save(user);
        return userMapper.toResponse(savedUser); // Chuyển đổi Entity đã lưu
sang Response DTO.
    }
}

```

```

@Override
public UserResponse updateUser(Long id, @Valid UpdateUserRequest request)
{
    User user = userRepository.findById(id)
        .orElseThrow(() -> new UserNotFoundException(id));

    userMapper.updateEntity(request, user); // Cập nhật Entity từ UpdateRequest DTO.

    User updatedUser = userRepository.save(user);
    return userMapper.toResponse(updatedUser);
}

@Override
@Transactional(readOnly = true) // Phương thức chỉ đọc, tối ưu hóa transaction.
public UserResponse getUserById(Long id) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new UserNotFoundException(id));

    return userMapper.toResponse(user);
}

@Override
@Transactional(readOnly = true)
public Page<UserResponse> getUsers(String name, String status, Pageable pageable) {
    Specification<User> spec = Specification.where(null); // Khởi tạo Specification.

    if (name != null && !name.trim().isEmpty()) {
        spec = spec.and(UserSpecifications.hasNameContaining(name)); // Thêm điều kiện tìm kiếm theo tên.
    }

    if (status != null && !status.trim().isEmpty()) {
        spec = spec.and(UserSpecifications.hasStatus(status)); // Thêm điều kiện tìm kiếm theo trạng thái.
    }

    Page<User> users = userRepository.findAll(spec, pageable); // Thực hiện truy vấn với Specification và phân trang.
    return users.map(userMapper::toResponse); // Chuyển đổi Page<User> sang Page<UserResponse>.
}

@Override
public void deleteUser(Long id) {

```

```

        if (!userRepository.existsById(id)) {
            throw new UserNotFoundException(id);
        }

        userRepository.deleteById(id);
    }

    @Override
    public void activateUser(Long id) {
        updateUserStatus(id, "ACTIVE");
    }

    @Override
    public void deactivateUser(Long id) {
        updateUserStatus(id, "INACTIVE");
    }

    private void updateUserStatus(Long id, String status) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));

        user.setStatus(status);
        userRepository.save(user);
    }
}

```

Giải thích:

- **DTOs:** Giúp tách biệt dữ liệu truyền tải khỏi Entity, tránh việc lộ thông tin nhạy cảm của Entity ra bên ngoài API và cho phép kiểm soát chặt chẽ dữ liệu đầu vào/đầu ra.
- **Service Interface:** Định nghĩa rõ ràng các chức năng mà Service cung cấp, giúp tăng tính module hóa và dễ dàng thay đổi implementation.
- **@Service** : Đánh dấu lớp là một Service component trong Spring, cho phép Spring tự động quản lý vòng đời của nó và inject các dependency khác.
- **@Transactional** : Đảm bảo rằng tất cả các thao tác trong phương thức Service được thực hiện trong một transaction duy nhất. Nếu có bất kỳ lỗi nào xảy ra, toàn bộ transaction sẽ được rollback.
- **@Validated** : Kích hoạt validation cho các tham số phương thức của Service. Khi kết hợp với **@Valid** trên DTO, nó sẽ tự động kiểm tra các ràng buộc validation được định nghĩa

trong DTO.

- **UserMapper** : Một thành phần quan trọng để chuyển đổi giữa các đối tượng Entity và DTO. Việc sử dụng Mapper giúp giữ cho Service Layer sạch sẽ, không bị lẫn lộn với logic chuyển đổi dữ liệu.
- **Specification** : Được sử dụng trong phương thức `getUsers` để xây dựng các truy vấn động dựa trên nhiều tiêu chí tìm kiếm. Nó cho phép bạn kết hợp các điều kiện tìm kiếm một cách linh hoạt.

7.3.2 Mapper Pattern

Mapper Pattern là một kỹ thuật phổ biến trong các ứng dụng Java để chuyển đổi giữa các đối tượng có cấu trúc khác nhau, đặc biệt là giữa Entity (mô hình dữ liệu của cơ sở dữ liệu) và DTO (Data Transfer Object, mô hình dữ liệu cho API hoặc giao diện người dùng). Việc sử dụng Mapper giúp tách biệt các mối quan tâm, giữ cho Service Layer sạch sẽ và dễ bảo trì hơn.

Các thư viện Mapper phổ biến bao gồm MapStruct, ModelMapper, và Dozer. Trong ví dụ này, chúng ta sẽ sử dụng MapStruct vì nó tạo ra code chuyển đổi tại thời điểm biên dịch, mang lại hiệu suất tốt và an toàn kiểu.

Dependencies (cho MapStruct):

XML

```
<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct</artifactId>
  <version>1.5.5.Final</version> <!-- Sử dụng phiên bản mới nhất -->
</dependency>
<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct-processor</artifactId>
  <version>1.5.5.Final</version>
  <scope>provided</scope>
</dependency>
```

Java

```

@Mapper(componentModel = "spring") // Đánh dấu interface này là một MapStruct
Mapper. componentModel = "spring" để MapStruct tạo ra một Spring component,
cho phép bạn inject nó vào các Service.
public interface UserMapper {

    // Chuyển đổi CreateUserRequest DTO sang User Entity.
    // MapStruct sẽ tự động ánh xạ các trường có cùng tên và kiểu.
    User toEntity(CreateUserRequest request);

    // Chuyển đổi User Entity sang UserResponse DTO.
    UserResponse toResponse(User user);

    // Cập nhật một User Entity hiện có từ UpdateUserRequest DTO.
    // @Mapping(target = "fieldName", ignore = true) để bỏ qua việc ánh xạ
    một trường cụ thể.
    // @MappingTarget User user: Chỉ định rằng đối tượng 'user' là đối tượng
    đích sẽ được cập nhật.
    @Mapping(target = "id", ignore = true) // Không cập nhật ID.
    @Mapping(target = "email", ignore = true) // Không cập nhật email (thường
    không cho phép thay đổi email).
    @Mapping(target = "status", ignore = true) // Không cập nhật trạng thái
    qua DTO này.
    @Mapping(target = "createdDate", ignore = true) // Không cập nhật ngày
    tạo.
    @Mapping(target = "lastModifiedDate", ignore = true) // Không cập nhật
    ngày sửa cuối cùng (sẽ được Auditing tự động xử lý).
    void updateEntity(UpdateUserRequest request, @MappingTarget User user);

    // Chuyển đổi danh sách User Entity sang danh sách UserResponse DTO.
    List<UserResponse> toResponseList(List<User> users);
}

```

Giải thích:

- **@Mapper(componentModel = "spring")** : Annotation chính của MapStruct.
componentModel = "spring" chỉ thị cho MapStruct tạo ra một implementation của **UserMapper** như một Spring Bean, cho phép bạn sử dụng **@Autowired** để inject nó vào các lớp Service.
- **toEntity(CreateUserRequest request)** : Phương thức này sẽ tạo một **User** Entity mới từ **CreateUserRequest** DTO. MapStruct sẽ tự động ánh xạ các trường có cùng tên và kiểu dữ liệu.

- **toResponse(User user)** : Phương thức này sẽ tạo một `UserResponse` DTO từ một `User` Entity.
- **updateEntity(UpdateUserRequest request, @MappingTarget User user)** : Đây là một phương thức cập nhật. Thay vì tạo một đối tượng mới, nó sẽ cập nhật các thuộc tính của đối tượng `user` hiện có dựa trên dữ liệu từ `UpdateUserRequest` . `@MappingTarget` chỉ định đối tượng đích.
- **@Mapping(target = "fieldName", ignore = true)** : Annotation này được sử dụng để chỉ thị cho MapStruct bỏ qua việc ánh xạ một trường cụ thể. Điều này rất hữu ích khi bạn không muốn một trường nào đó bị thay đổi thông qua DTO (ví dụ: ID, email, hoặc các trường auditing).
- **List<UserResponse> toResponseList(List<User> users)** : MapStruct cũng hỗ trợ ánh xạ các danh sách một cách tự động.
- **Lợi ích của Mapper Pattern:**
 - **Tách biệt mối quan tâm:** Giữ cho Entity sạch sẽ, chỉ tập trung vào việc ánh xạ với cơ sở dữ liệu. DTO tập trung vào việc truyền dữ liệu giữa các lớp.
 - **Bảo mật:** Bạn có thể kiểm soát chính xác dữ liệu nào được hiển thị hoặc chấp nhận từ client, tránh việc lộ các trường nhạy cảm của Entity.
 - **Linh hoạt:** Dễ dàng thay đổi cấu trúc API (DTO) mà không ảnh hưởng đến cấu trúc cơ sở dữ liệu (Entity) và ngược lại.
 - **Kiểm thử:** Dễ dàng kiểm thử Service Layer mà không cần phụ thuộc vào Controller hoặc các thành phần giao diện người dùng.

7.4 Testing Strategies

Kiểm thử (Testing) là một phần không thể thiếu trong quá trình phát triển phần mềm, giúp đảm bảo chất lượng, độ tin cậy và tính đúng đắn của ứng dụng. Trong một ứng dụng Spring Data JPA, chúng ta thường thực hiện các loại kiểm thử sau:

7.4.1 Repository Testing

Kiểm thử Repository tập trung vào việc kiểm tra các lớp Repository, đảm bảo rằng chúng tương tác đúng với cơ sở dữ liệu và các phương thức truy vấn hoạt động như mong đợi. Spring Boot cung cấp `@DataJpaTest` để hỗ trợ kiểm thử các thành phần JPA một cách hiệu quả.

Java

```
@DataJpaTest // Annotation này cấu hình một slice test cho các thành phần JPA. Nó tự động cấu hình một cơ sở dữ liệu nhúng (in-memory database) như H2 và chỉ tải các bean liên quan đến JPA.
class UserRepositoryTest {

    @Autowired
    private TestEntityManager entityManager; // TestEntityManager là một EntityManager được cấu hình đặc biệt cho các bài kiểm thử, cho phép bạn thực hiện các thao tác cơ sở dữ liệu trực tiếp để thiết lập dữ liệu kiểm thử.

    @Autowired
    private UserRepository userRepository; // Inject Repository mà bạn muốn kiểm thử.

    @Test // Đánh dấu đây là một phương thức kiểm thử JUnit.
    void testFindByEmail() {
        // Given: Thiết lập dữ liệu ban đầu cho bài kiểm thử.
        User user = new User("John Doe", "john@example.com");
        entityManager.persistAndFlush(user); // Lưu Entity vào DB và flush ngay lập tức để đảm bảo dữ liệu có sẵn cho truy vấn.

        // When: Thực hiện hành động mà bạn muốn kiểm thử.
        User found = userRepository.findByEmail("john@example.com"); // Gọi phương thức Repository cần kiểm thử.

        // Then: Xác nhận kết quả mong đợi.
        assertThat(found).isNotNull(); // Sử dụng AssertJ để kiểm tra kết quả.
        assertThat(found.getName()).isEqualTo("John Doe");
    }

    @Test
    void testFindByStatus() {
        // Given
        User activeUser = new User("Active User", "active@example.com");
        activeUser.setStatus("ACTIVE");

        User inactiveUser = new User("Inactive User", "inactive@example.com");
```

```

        inactiveUser.setStatus("INACTIVE");

        entityManager.persist(activeUser);
        entityManager.persist(inactiveUser);
        entityManager.flush(); // Đảm bảo cả hai user đều được lưu vào DB.

        // When
        List<User> activeUsers = userRepository.findByStatus("ACTIVE");

        // Then
        assertThat(activeUsers).hasSize(1);
        assertThat(activeUsers.get(0).getName()).isEqualTo("Active User");
    }

    @Test
    void testCustomQuery() {
        // Given
        User user1 = new User("John", "john@example.com");
        user1.setAge(25);

        User user2 = new User("Jane", "jane@example.com");
        user2.setAge(30);

        entityManager.persist(user1);
        entityManager.persist(user2);
        entityManager.flush();

        // When
        List<User> usersOlderThan25 = userRepository.findUsersOlderThan(25);

        // Then
        assertThat(usersOlderThan25).hasSize(1);
        assertThat(usersOlderThan25.get(0).getName()).isEqualTo("Jane");
    }
}

```

Giải thích:

- **@DataJpaTest** : Đây là một annotation kiểm thử chuyên biệt của Spring Boot. Nó chỉ tải các cấu hình liên quan đến JPA (DataSource, EntityManager, Spring Data JPA Repositories) và không tải toàn bộ ứng dụng. Điều này giúp các bài kiểm thử chạy nhanh hơn.
- Nó tự động cấu hình một cơ sở dữ liệu nhúng (in-memory database) như H2, giúp các bài kiểm thử độc lập và không ảnh hưởng đến cơ sở dữ liệu phát triển/sản xuất.

- Mặc định, các bài kiểm thử `@DataJpaTest` là transactional và sẽ rollback sau mỗi bài kiểm thử, đảm bảo môi trường sạch sẽ cho bài kiểm thử tiếp theo.
- **TestEntityManager** : Một phiên bản của `EntityManager` được cung cấp bởi Spring Test. Nó cho phép bạn thực hiện các thao tác cơ sở dữ liệu trực tiếp (như `persist` , `find` , `remove`) để thiết lập và xác minh dữ liệu kiểm thử. Nó rất hữu ích để đảm bảo dữ liệu kiểm thử được chuẩn bị chính xác trước khi gọi các phương thức Repository.
- **@Autowired** : Được sử dụng để inject các bean mà Spring Boot đã cấu hình cho bài kiểm thử (ví dụ: `TestEntityManager` , `UserRepository`).
- **Cấu trúc Given-When-Then**: Một mẫu phổ biến để viết các bài kiểm thử, giúp chúng dễ đọc và dễ hiểu:
 - **Given**: Thiết lập trạng thái ban đầu hoặc dữ liệu cần thiết cho bài kiểm thử.
 - **When**: Thực hiện hành động hoặc gọi phương thức mà bạn muốn kiểm thử.
 - **Then**: Xác nhận rằng kết quả của hành động đó khớp với mong đợi.
- **entityManager.persistAndFlush(entity)** : Lưu một Entity vào cơ sở dữ liệu và ngay lập tức đồng bộ hóa (flush) các thay đổi vào DB. Điều này đảm bảo rằng Entity có sẵn trong DB cho các truy vấn tiếp theo trong cùng một bài kiểm thử.
- **assertThat(...)** : Sử dụng thư viện AssertJ để viết các khẳng định (assertions) một cách rõ ràng và dễ đọc. AssertJ cung cấp một API fluent cho các khẳng định.

7.4.2 Service Testing

Kiểm thử Service Layer tập trung vào việc kiểm tra logic nghiệp vụ của ứng dụng, đảm bảo rằng các phương thức Service hoạt động đúng đắn, xử lý dữ liệu và tương tác với các thành phần khác (như Repository, Mapper) một cách chính xác. Đối với kiểm thử Service, chúng ta thường sử dụng Mockito để mock các dependency bên ngoài (ví dụ: Repository, Mapper) để tập trung kiểm thử logic của Service mà không cần tương tác với cơ sở dữ liệu thực.

Java

```
@ExtendWith(MockitoExtension.class) // Kích hoạt Mockito cho JUnit 5.
class UserServiceTest {
```

```
@Mock // Tạo một mock object cho UserRepository. Các phương thức của
userRepository sẽ không thực sự gọi đến DB mà sẽ được kiểm soát bởi Mockito.
private UserRepository userRepository;

@Mock // Tạo một mock object cho UserMapper.
private UserMapper userMapper;

@InjectMocks // Inject các mock object (userRepository, userMapper) vào
instance của UserServiceImpl.
private UserServiceImpl userService;

@Test
void testCreateUser_Success() {
    // Given: Thiết lập dữ liệu đầu vào và hành vi mong muốn của các mock
    object.

    CreateUserRequest request = new CreateUserRequest();
    request.setName("John Doe");
    request.setEmail("john@example.com");
    request.setAge(25);

    User user = new User(); // Entity sẽ được tạo từ DTO.
    user.setName("John Doe");
    user.setEmail("john@example.com");
    user.setAge(25);

    User savedUser = new User(); // Entity sau khi được lưu.
    savedUser.setId(1L);
    savedUser.setName("John Doe");
    savedUser.setEmail("john@example.com");
    savedUser.setAge(25);
    savedUser.setStatus("ACTIVE");

    UserResponse response = new UserResponse(); // DTO sẽ được trả về từ
    Service.
    response.setId(1L);
    response.setName("John Doe");
    response.setEmail("john@example.com");
    response.setAge(25);
    response.setStatus("ACTIVE");

    // Định nghĩa hành vi của các mock object khi các phương thức của
    chúng được gọi.

    when(userRepository.existsByEmail("john@example.com")).thenReturn(false); //
    Khi kiểm tra email, trả về false (không tồn tại).
    when(userMapper.toEntity(request)).thenReturn(user); // Khi chuyển
    đổi DTO sang Entity, trả về user Entity.
    when(userRepository.save(any(User.class))).thenReturn(savedUser); //
```

Khi lưu bất kỳ user nào, trả về savedUser.

```
when(userMapper.toResponse(savedUser)).thenReturn(response); // Khi chuyển đổi Entity sang Response DTO, trả về response DTO.
```

// When: Gọi phương thức Service cần kiểm thử.

```
UserResponse result = userService.createUser(request);
```

// Then: Xác nhận kết quả và kiểm tra xem các phương thức mock có được gọi đúng cách không.

```
assertThat(result).isNotNull();
```

```
assertThat(result.getId()).isEqualTo(1L);
```

```
assertThat(result.getName()).isEqualTo("John Doe");
```

```
assertThat(result.getStatus()).isEqualTo("ACTIVE");
```

verify(userRepository).existsByEmail("john@example.com"); // Xác minh rằng phương thức existsByEmail đã được gọi.

verify(userRepository).save(any(User.class)); // Xác minh rằng phương thức save đã được gọi.

```
}
```

@Test

```
void testCreateUser_DuplicateEmail() {
```

// Given

```
CreateUserRequest request = new CreateUserRequest();
```

```
request.setEmail("john@example.com");
```

```
when(userRepository.existsByEmail("john@example.com")).thenReturn(true); // Giả lập email đã tồn tại.
```

// When & Then: Kiểm tra xem ngoại lệ có được ném ra đúng cách không.

```
assertThatThrownBy(() -> userService.createUser(request))
```

```
.isInstanceOf(DuplicateEmailException.class) // Xác nhận loại
```

ngoại lệ.

```
.hasMessage("User with email already exists: john@example.com");
```

// Xác nhận thông báo lỗi.

verify(userRepository).existsByEmail("john@example.com"); // Xác minh rằng existsByEmail đã được gọi.

verify(userRepository, never()).save(any(User.class)); // Xác minh rằng save không bao giờ được gọi (vì email đã trùng).

```
}
```

@Test

```
void testGetUserById_NotFound() {
```

// Given

```
Long userId = 1L;
```

```
when(userRepository.findById(userId)).thenReturn(Optional.empty());
```

```
// Giả lập không tìm thấy user.

// When & Then
assertThatThrownBy(() -> userService.getUserById(userId))
    .assertInstanceOf(UserNotFoundException.class)
    .hasMessage("User not found with ID: 1");
}
}
```

Giải thích:

- **@ExtendWith(MockitoExtension.class)** : Kích hoạt Mockito cho các bài kiểm thử JUnit 5. Nó cho phép bạn sử dụng các annotation của Mockito như **@Mock** và **@InjectMocks** .
- **@Mock** : Tạo một mock object cho một dependency. Mock object là một phiên bản giả lập của một lớp, cho phép bạn định nghĩa hành vi của nó và kiểm tra xem các phương thức của nó có được gọi hay không. Điều này giúp cô lập lớp đang được kiểm thử khỏi các dependency thực tế.
- **@InjectMocks** : Inject các mock object đã tạo (**@Mock**) vào instance của lớp đang được kiểm thử (**UserServiceImpl**). Mockito sẽ cố gắng inject các mock object vào các trường hoặc constructor của lớp này.
- **when(mock.method()).thenReturn(value)** : Định nghĩa hành vi của một phương thức trên mock object. Khi phương thức **method()** của **mock** được gọi, nó sẽ trả về **value** đã định nghĩa.
- **verify(mock).method()** : Xác minh rằng một phương thức trên mock object đã được gọi. Bạn cũng có thể sử dụng **verify(mock, times(n)).method()** để kiểm tra số lần gọi, hoặc **verify(mock, never()).method()** để kiểm tra xem phương thức có bị gọi hay không.
- **any(Class.class)** : Một đối số khớp (argument matcher) của Mockito. Nó cho phép bạn khớp với bất kỳ đối tượng nào thuộc một kiểu cụ thể khi định nghĩa hành vi hoặc xác minh cuộc gọi phương thức.
- **assertThatThrownBy(() -> ...).assertInstanceOf(...).hasMessage(...)** : Sử dụng AssertJ để kiểm tra xem một ngoại lệ có được ném ra đúng cách hay không, loại ngoại lệ là gì và thông báo lỗi có khớp không.

- **Lợi ích của Service Testing với Mockito:**

- **Cô lập:** Chỉ kiểm thử logic của Service, không phụ thuộc vào cơ sở dữ liệu hoặc các hệ thống bên ngoài.
- **Tốc độ:** Các bài kiểm thử chạy rất nhanh vì không có tương tác DB thực tế.
- **Kiểm soát:** Bạn có toàn quyền kiểm soát hành vi của các dependency, giúp kiểm thử các kịch bản phức tạp (ví dụ: lỗi DB, dữ liệu không tồn tại) một cách dễ dàng.

7.4.3 Integration Testing

Kiểm thử tích hợp (Integration Testing) kiểm tra sự tương tác giữa các thành phần khác nhau của ứng dụng (ví dụ: Controller, Service, Repository và cơ sở dữ liệu thực). Mục tiêu là đảm bảo rằng các thành phần này hoạt động cùng nhau một cách chính xác. Spring Boot cung cấp `@SpringBootTest` để hỗ trợ kiểm thử tích hợp.

Java

```
@SpringBootTest // Tải toàn bộ Spring application context, bao gồm tất cả các bean và cấu hình.
@Transactional // Đảm bảo mỗi bài kiểm thử chạy trong một transaction riêng và rollback sau khi hoàn thành để giữ cho DB sạch.
@TestPropertySource(properties = {
    "spring.datasource.url=jdbc:h2:mem:testdb", // Cấu hình sử dụng cơ sở dữ liệu H2 trong bộ nhớ cho kiểm thử.
    "spring.jpa.hibernate.ddl-auto=create-drop" // Tự động tạo và xóa schema cơ sở dữ liệu cho mỗi lần chạy kiểm thử.
})
class UserServiceIntegrationTest {

    @Autowired
    private UserService userService; // Inject Service để kiểm thử logic nghiệp vụ.

    @Autowired
    private UserRepository userRepository; // Inject Repository để kiểm tra trực tiếp dữ liệu trong DB.

    @Test
    void testCreateAndRetrieveUser() {
        // Given: Chuẩn bị dữ liệu đầu vào.
        CreateUserRequest request = new CreateUserRequest();
        request.setName("Integration Test User");
    }
}
```



```

request.setEmail("integration@example.com");
request.setAge(30);

// When: Thực hiện các thao tác thông qua Service.
UserResponse created = userService.createUser(request);
UserResponse retrieved = userService.getUserById(created.getId());

// Then: Xác nhận kết quả thông qua Service và kiểm tra trực tiếp
trong DB.
assertThat(retrieved).isNotNull();
assertThat(retrieved.getName()).isEqualTo("Integration Test User");

assertThat(retrieved.getEmail()).isEqualTo("integration@example.com");
assertThat(retrieved.getAge()).isEqualTo(30);
assertThat(retrieved.getStatus()).isEqualTo("ACTIVE");

// Kiểm tra trực tiếp trong DB để đảm bảo dữ liệu đã được lưu đúng.
assertThat(userRepository.findByEmail("integration@example.com")).isNotNull();
;
}

@Test
void testUserLifecycle() {
    // Create
    CreateUserRequest createRequest = new CreateUserRequest();
    createRequest.setName("Lifecycle User");
    createRequest.setEmail("lifecycle@example.com");
    createRequest.setAge(25);

    UserResponse created = userService.createUser(createRequest);
    assertThat(created.getStatus()).isEqualTo("ACTIVE");

    // Update
    UpdateUserRequest updateRequest = new UpdateUserRequest();
    updateRequest.setName("Updated Lifecycle User");
    updateRequest.setAge(26);

    UserResponse updated = userService.updateUser(created.getId(),
updateRequest);
    assertThat(updated.getName()).isEqualTo("Updated Lifecycle User");
    assertThat(updated.getAge()).isEqualTo(26);

    // Deactivate
    userService.deactivateUser(created.getId());
    UserResponse deactivated = userService.getUserById(created.getId());
    assertThat(deactivated.getStatus()).isEqualTo("INACTIVE");
}

```

```

// Delete
userService.deleteUser(created.getId());
assertThatThrownBy(() -> userService.getUserById(created.getId()))
    .isInstanceOf(UserNotFoundException.class);
}
}

```

Giải thích:

- **@SpringBootTest** : Annotation này tải toàn bộ Spring application context, bao gồm tất cả các bean và cấu hình của ứng dụng. Điều này làm cho các bài kiểm thử tích hợp chậm hơn so với kiểm thử đơn vị, nhưng chúng cung cấp sự đảm bảo cao hơn về việc các thành phần hoạt động cùng nhau một cách chính xác.
- Bạn có thể cấu hình `webEnvironment` của `@SpringBootTest` để kiểm soát cách ứng dụng web được khởi động (ví dụ: `WebEnvironment.MOCK` để sử dụng `MockMvc`, `WebEnvironment.RANDOM_PORT` để khởi động một server thực trên một cổng ngẫu nhiên).
- **@Transactional** : Đảm bảo rằng mỗi bài kiểm thử chạy trong một transaction riêng biệt và transaction đó sẽ được rollback sau khi bài kiểm thử hoàn thành. Điều này giúp giữ cho cơ sở dữ liệu sạch sẽ giữa các bài kiểm thử, đảm bảo tính độc lập của chúng.
- **@TestPropertySource** : Cho phép bạn ghi đè các thuộc tính cấu hình trong quá trình kiểm thử. Ở đây, chúng ta cấu hình sử dụng cơ sở dữ liệu H2 trong bộ nhớ và `ddl-auto=create-drop` để đảm bảo môi trường kiểm thử sạch sẽ cho mỗi lần chạy.
- **@Autowired** : Được sử dụng để inject các bean thực tế của ứng dụng (ví dụ: `UserService` , `UserRepository`) vào lớp kiểm thử.
- **Cấu trúc Given-When-Then**: Một mẫu phổ biến để viết các bài kiểm thử, giúp chúng dễ đọc và dễ hiểu:
 - **Given**: Thiết lập trạng thái ban đầu hoặc dữ liệu cần thiết cho bài kiểm thử.
 - **When**: Thực hiện hành động hoặc gọi phương thức mà bạn muốn kiểm thử.
 - **Then**: Xác nhận rằng kết quả của hành động đó khớp với mong đợi.
- **Lợi ích của Integration Testing**:

- **Kiểm tra luồng end-to-end:** Đảm bảo rằng tất cả các thành phần (Controller, Service, Repository, DB) hoạt động cùng nhau một cách chính xác.
- **Phát hiện lỗi tích hợp:** Giúp tìm ra các vấn đề phát sinh khi các module khác nhau được kết nối.
- **Đảm bảo tính đúng đắn của API:** Xác minh rằng các endpoint API trả về phản hồi đúng định dạng và mã trạng thái HTTP phù hợp. ``

7.5 Performance Best Practices

Để đảm bảo ứng dụng Spring Data JPA hoạt động hiệu quả và có khả năng mở rộng, việc tối ưu hóa hiệu suất là rất quan trọng. Dưới đây là một số best practices và cấu hình liên quan đến hiệu suất.

7.5.1 Database Connection Pool

Connection Pooling là một kỹ thuật quan trọng để quản lý hiệu quả các kết nối cơ sở dữ liệu. Thay vì mở và đóng một kết nối mới cho mỗi yêu cầu, Connection Pool duy trì một tập hợp các kết nối sẵn sàng để sử dụng lại, giúp giảm đáng kể chi phí overhead và cải thiện hiệu suất. HikariCP là một trong những Connection Pool nhanh nhất và được khuyến nghị sử dụng trong Spring Boot.

Plain Text

```
# HikariCP configuration
spring.datasource.hikari.maximum-pool-size=20 // Số lượng kết nối tối đa
trong pool. Nên điều chỉnh dựa trên số lượng CPU core và loại workload (CPU-
bound vs I/O-bound).
spring.datasource.hikari.minimum-idle=5 // Số lượng kết nối nhàn rỗi tối
thiểu được duy trì trong pool. Giúp tránh việc tạo kết nối mới khi có nhu cầu
tăng đột biến.
spring.datasource.hikari.idle-timeout=300000 // Thời gian (ms) mà một kết nối
có thể nhàn rỗi trong pool trước khi bị đóng. Mặc định là 10 phút (600000
ms).
spring.datasource.hikari.max-lifetime=600000 // Thời gian (ms) tối đa mà một
kết nối có thể tồn tại trong pool. Sau thời gian này, kết nối sẽ bị đóng và
được thay thế bằng một kết nối mới. Giúp tránh các vấn đề với kết nối bị hỏng
hoặc hết hạn.
spring.datasource.hikari.connection-timeout=20000 // Thời gian (ms) tối đa để
chờ một kết nối từ pool. Nếu không có kết nối nào khả dụng trong thời gian
này, một SQLException sẽ được ném ra.
```

`spring.datasource.hikari.leak-detection-threshold=60000` // Thời gian (ms) mà một kết nối có thể được sử dụng trước khi một cảnh báo về rò rỉ kết nối được ghi lại. Giúp phát hiện các trường hợp kết nối không được đóng đúng cách.

Connection validation

`spring.datasource.hikari.connection-test-query=SELECT 1` // Một truy vấn SQL đơn giản được sử dụng để kiểm tra tính hợp lệ của kết nối trước khi trả về cho ứng dụng. Đảm bảo kết nối vẫn hoạt động.

`spring.datasource.hikari.validation-timeout=3000` // Thời gian (ms) tối đa để chờ truy vấn kiểm tra kết nối hoàn thành.

Giải thích:

- **HikariCP** : Là Connection Pool mặc định và được khuyến nghị trong Spring Boot vì hiệu suất vượt trội của nó. Các thuộc tính trên được sử dụng để cấu hình hành vi của HikariCP.
- **maximum-pool-size** : Đây là một trong những cấu hình quan trọng nhất. Giá trị này phụ thuộc vào nhiều yếu tố như số lượng CPU cores, loại workload (CPU-bound hay I/O-bound), và khả năng của cơ sở dữ liệu. Một quy tắc chung là $(\text{số lượng CPU cores} * 2) + 1$ cho các ứng dụng CPU-bound, và có thể cao hơn cho các ứng dụng I/O-bound.
- **minimum-idle** : Đảm bảo rằng luôn có một số lượng kết nối nhất định sẵn sàng trong pool, giúp giảm độ trễ khi có yêu cầu mới.
- **idle-timeout** và **max-lifetime** : Giúp quản lý vòng đời của các kết nối, đóng các kết nối không sử dụng hoặc đã tồn tại quá lâu để tránh các vấn đề như kết nối bị hỏng hoặc hết hạn từ phía cơ sở dữ liệu.
- **connection-timeout** : Ngăn chặn ứng dụng bị treo vô thời hạn khi không thể lấy được kết nối từ pool.
- **leak-detection-threshold** : Một tính năng hữu ích để phát hiện các trường hợp ứng dụng không đóng kết nối đúng cách, dẫn đến rò rỉ tài nguyên.
- **connection-test-query** : Đảm bảo rằng kết nối được trả về từ pool vẫn còn hoạt động và có thể được sử dụng để tương tác với cơ sở dữ liệu. Điều này đặc biệt quan trọng trong các môi trường mạng không ổn định.

7.5.2 JPA Performance Settings

Ngoài việc cấu hình Connection Pool, bạn cũng có thể tối ưu hóa hiệu suất của JPA/Hibernate thông qua các thuộc tính cấu hình. Các cài đặt này ảnh hưởng đến cách Hibernate tương tác với cơ sở dữ liệu, từ việc thực hiện các thao tác batch đến quản lý cache.

Plain Text

```
# Batch processing: Cấu hình để Hibernate thực hiện các thao tác INSERT,
UPDATE, DELETE theo lô (batch) thay vì từng câu lệnh riêng lẻ. Điều này giúp
giảm số lượng round-trip đến cơ sở dữ liệu, cải thiện đáng kể hiệu suất cho
các thao tác dữ liệu lớn.
spring.jpa.properties.hibernate.jdbc.batch_size=20 // Số lượng câu lệnh SQL
được nhóm lại trong một batch. Giá trị tối ưu phụ thuộc vào cơ sở dữ liệu và
kích thước của các bản ghi.
spring.jpa.properties.hibernate.order_inserts=true // Sắp xếp các câu lệnh
INSERT theo thứ tự để tối ưu hóa việc sử dụng batching.
spring.jpa.properties.hibernate.order_updates=true // Sắp xếp các câu lệnh
UPDATE theo thứ tự để tối ưu hóa việc sử dụng batching.
spring.jpa.properties.hibernate.jdbc.batch_versioned_data=true // Cho phép
batching cho các Entity có versioning (ví dụ: sử dụng @Version cho Optimistic
Locking).

# Query optimization: Các cài đặt liên quan đến việc tối ưu hóa truy vấn và
quản lý kế hoạch truy vấn.
spring.jpa.properties.hibernate.query.plan_cache_max_size=2048 // Kích thước
tối đa của cache cho các kế hoạch truy vấn đã được biên dịch. Giúp tái sử
dụng các kế hoạch truy vấn, tránh việc biên dịch lại.
spring.jpa.properties.hibernate.query.plan_parameter_metadata_max_size=128 //
Kích thước tối đa của cache cho metadata tham số của kế hoạch truy vấn.

# Statistics: Kích hoạt thu thập thống kê hiệu suất của Hibernate. Rất hữu
ích cho việc phân tích và tối ưu hóa.
spring.jpa.properties.hibernate.generate_statistics=true // Kích hoạt việc
thu thập thống kê về các hoạt động của Hibernate (ví dụ: số lượng truy vấn,
thời gian thực thi, cache hits/misses).
spring.jpa.properties.hibernate.session.events.log.LOG_QUERIES_SLOWER_THAN_MS
=100 // Ghi log các truy vấn SQL có thời gian thực thi vượt quá ngưỡng (ms)
đã định. Hữu ích để phát hiện các truy vấn chậm.
```

Giải thích:

- **Batch Processing:** Khi bạn thực hiện nhiều thao tác `save()` , `update()` , `delete()` liên tiếp, Hibernate có thể nhóm chúng lại thành một batch và gửi đến cơ sở dữ liệu trong một lần duy nhất. Điều này giảm số lượng round-trip mạng và cải thiện hiệu suất đáng

kể. Các thuộc tính `batch_size` , `order_inserts` , `order_updates` giúp tối ưu hóa quá trình này.

- **Query Plan Cache:** Hibernate có thể cache các kế hoạch truy vấn đã được biên dịch. Khi một truy vấn tương tự được thực hiện lại, Hibernate có thể sử dụng lại kế hoạch đã có thay vì biên dịch lại, giúp tăng tốc độ thực thi truy vấn.
- **Statistics:** Việc kích hoạt thống kê của Hibernate là một công cụ mạnh mẽ để theo dõi hiệu suất. Bạn có thể xem các số liệu thống kê như số lượng truy vấn, thời gian thực thi trung bình, tỷ lệ cache hit/miss, v.v. Điều này giúp bạn xác định các điểm nghẽn và các truy vấn cần tối ưu hóa. `LOG_QUERIES_SLOWER_THAN_MS` đặc biệt hữu ích để nhanh chóng phát hiện các truy vấn chậm trong môi trường phát triển hoặc staging.

7.5.3 Monitoring và Logging

Giám sát (Monitoring) và ghi log (Logging) là các hoạt động thiết yếu để theo dõi hiệu suất, phát hiện vấn đề và gỡ lỗi trong các ứng dụng sản xuất. Đối với Spring Data JPA, việc cấu hình logging chi tiết cho SQL và Hibernate, cùng với việc thu thập các số liệu thống kê, sẽ cung cấp cái nhìn sâu sắc về hoạt động của cơ sở dữ liệu.

Plain Text

```
# SQL logging: Cấu hình mức độ log cho các câu lệnh SQL được Hibernate tạo ra
và các tham số của chúng.
logging.level.org.hibernate.SQL=DEBUG // Ghi log tất cả các câu lệnh SQL được
Hibernate thực thi.
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE // Ghi log
các giá trị tham số được bind vào câu lệnh SQL. Rất hữu ích để gỡ lỗi các
truy vấn.
logging.level.org.hibernate.stat=DEBUG // Ghi log các thống kê hiệu suất của
Hibernate (ví dụ: số lượng truy vấn, thời gian thực thi).

# Performance logging: Cấu hình mức độ log cho các thành phần liên quan đến
transaction và JPA của Spring.
logging.level.org.springframework.transaction=DEBUG // Ghi log các sự kiện
liên quan đến quản lý transaction của Spring.
logging.level.org.springframework.orm.jpa=DEBUG // Ghi log các sự kiện liên
quan đến JPA của Spring.
```

Java

```

@Component // Đánh dấu lớp này là một Spring component.
public class DatabaseMetricsCollector {

    @Autowired
    private EntityManagerFactory entityManagerFactory; // Inject
    EntityManagerFactory để truy cập các thống kê của Hibernate.

    // Lắng nghe sự kiện SlowQueryEvent của Hibernate (nếu được cấu hình).
    @EventListener
    @Async // Thực thi phương thức này một cách bất đồng bộ để không làm chậm
    luồng chính.
    public void handleSlowQuery(SlowQueryEvent event) {
        // Ghi log các truy vấn chậm được phát hiện bởi Hibernate.
        log.warn("Slow query detected: {} ms - {}",
            event.getExecutionTime(), event.getQuery());
    }

    @Scheduled(fixedRate = 60000) // Lên lịch để phương thức này chạy mỗi 60
    giây (1 phút).
    public void collectStatistics() {
        // Lấy đối tượng Statistics từ SessionFactory của Hibernate.
        Statistics stats = entityManagerFactory.unwrap(SessionFactory.class)
            .getStatistics();

        // Ghi log các số liệu thống kê quan trọng của JPA/Hibernate.
        log.info("JPA Statistics - Queries: {}, Cache hits: {}, Cache misses:
        {}",
            stats.getQueryExecutionCount(), // Tổng số truy vấn đã thực thi.
            stats.getSecondLevelCacheHitCount(), // Số lần dữ liệu được tìm
            thấy trong Second Level Cache.
            stats.getSecondLevelCacheMissCount()); // Số lần dữ liệu không
            được tìm thấy trong Second Level Cache.
    }
}

```

Giải thích:

- **SQL Logging:** Việc bật log cho `org.hibernate.SQL` ở mức `DEBUG` sẽ hiển thị tất cả các câu lệnh SQL được thực thi. Kết hợp với `org.hibernate.type.descriptor.sql.BasicBinder` ở mức `TRACE`, bạn sẽ thấy cả các giá trị tham số được bind vào câu lệnh SQL, điều này cực kỳ hữu ích cho việc gỡ lỗi và tối ưu hóa truy vấn.
- **Performance Logging:** Các mức log cho `org.springframework.transaction` và `org.springframework.orm.jpa` cung cấp thông tin chi tiết về quá trình quản lý transaction

và tương tác của Spring với JPA.

- **DatabaseMetricsCollector** : Một ví dụ về cách bạn có thể thu thập và ghi log các số liệu thống kê hiệu suất của Hibernate một cách định kỳ.
- **@EventListener** và **SlowQueryEvent** : Nếu bạn cấu hình `hibernate.session.events.log.LOG_QUERIES_SLOWER_THAN_MS` , Hibernate sẽ phát ra **SlowQueryEvent** khi một truy vấn vượt quá ngưỡng thời gian. Bạn có thể lắng nghe sự kiện này để ghi log hoặc gửi cảnh báo.
- **@Scheduled** : Cho phép bạn lên lịch các tác vụ định kỳ để thu thập và ghi log các số liệu thống kê từ **Hibernate Statistics** . Các số liệu này bao gồm tổng số truy vấn, số lần cache hit/miss, v.v., giúp bạn có cái nhìn tổng quan về hiệu suất của tầng persistence.